Project

For your course project, you will design and implement an end-to-end ML system. In your project, you must use the techniques discussed in our lectures to address some of the challenges *also* discussed in those lectures.

- Project context requirement
- Group work expectations
- Project deliverables and deadlines
  - Project proposal (Due Mar 2)
  - Initial implementation (Due Apr 6)
- Policy on AI use

Project context requirement

For your project, you will integrate one or more *complementary* ML features into an existing open-source, self-hosted software system that you will run on Chameleon.

Why? In practice, ML models most often operate as components within larger systems that impose constraints around data availability, latency, reliability, deployment, and operational ownership. If you design a new service "around the model" you get to ignore these constraints and do whatever is convenient, which bypasses the core challenges the course is intended to teach. So instead, we are asking you to design and implement a complementary feature in the context of an existing system and its constraints.

For example, you may design a feature that complements:

- meetings/webinars via Jitsi
- diagram sketching via Excalidraw
- audiobook/podcast library via audiobookshelf
- recipe manager via Mealie
- photo library via immich or PhotoPrism
- book library/reading with BookLore
- blog/publishing via Ghost
- team chat via Mattermost or Zulip
- media server via Jellyfin
- music streaming via Navidrome
- note taking via Trilium or DocMost

- document storage and sharing via NextCloud
- Internet search via SearXNG
- livestreaming via Owncast
- document management via Paperless-ngx
- caregiving via BabyBuddy
- project management via AppFlowy
- personal finance via ActualBudget
- travel planning via AdventureLog
- fitness tracking via SparkyFitness
- etc.

If the project you plan to complement has fewer than 2.5k stars on Github, you should get advance approval from course staff before preparing your proposal.

You don't have to use the project exactly as intended - for example, if you want to implement an ML feature for team chat that is specifically designed for students working on a group project together, you can do it in Zulip even if that's a general purpose chat service. Or if you want to implement an ML feature for a news website, you can do it with Ghost, and so on.

Also, integration with the core open source project is exempt from the "you must understand everything about your code and what it does" policy on AI use. You are welcome to vibe code the part that integrates with the core open source service. Right around the time when you'll be integrating your system (in April), we'll give some additional guidance and resources on using an AI coding agent to help with this part.

Additional requirements:

- Your ML feature must be designed so that when deployed in "production", you get new data and feedback from "users", and can use this for retraining.
- You can use an LLM out-of-the-box (without retraining) for part of your project, but if you do, you must also include another model that you train/retrain.
- You must use at least one high-quality non-synthetic external dataset with known lineage (who created it, how, etc.)

Group work expectations

You will complete these projects in groups of 3 or 4, where certain elements of the project are going to be “owned” by all group members, and other parts are going to be “owned” by individual group members.

Role	Responsibilities
All group members (joint)	Project idea and value proposition; high-level approach; overall system integration *(3-person team): Platform / DevOps responsibilities are shared; each member owns automation related to their primary role (Unit 3)*
Training	Model training and retraining pipelines (Units 5–6) Offline evaluation (part of Unit 8) Safeguarding elements related to role (Unit 10)
Serving	Model serving (Unit 7) Online evaluation and monitoring (part of Unit 8) Safeguarding elements related to role (Unit 10)
Data	Data pipeline (Unit 4) Closing the feedback loop (getting outcomes/labels in production) (part of Unit 8) Emulated operational data Safeguarding elements related to role (Unit 10)
DevOps / Platform (4-person team)	Infrastructure as code, CI/CD/CT pipelines, automation (Unit 3) Infrastructure monitoring and observability Safeguarding elements related to role (Unit 10)

Part of your project grade will be common to the entire group, based on the “jointly owned” elements and shared responsibilities; part of your project grade will be individual, based on the work you have produced in your personal role.

Can I work by myself and take on all of these roles? No, not in this course. An explicit learning objectives of this course is to practice building, operating, and integrating ML systems as a team activity. In real ML systems, components such as data pipelines, training workflows, serving infrastructure, and automation are developed independently and must interoperate through well-defined contracts. In a solo project, you can change interfaces arbitrarily to simplify implementation, which bypasses the core challenge of designing components to work as part of a whole. Therefore, a group project is required.

Project deliverables and deadlines

Milestone	Due Date	Points	Scope
Project proposal	Mar 2, 2026	5 / 40	Problem statement, data sources, modeling approach, alignment with business requirements
Initial implementation	Apr 6, 2026	10 / 40	Data, model training, model serving implemented individually (not necessarily integrated); overall pipeline with dummy steps also implemented for 4-person groups
System implementation	Apr 20, 2026	15 / 40	All components tightly integrated into a single end-to-end ML system, including safeguarding
Ongoing operation	May 4, 2026	10 / 40	Operation with emulated “live” data; operational behavior, stability, and evaluation over time

More specific information will be shared ahead of each deadline.

Project proposal (Due Mar 2)

Focus: intent, feasibility, business alignment.

Format: You will submit a document (max 2 pages) and slides for a presentation (10 minutes for a 3-person team, 12 minutes for a 4-person team) covering the items listed below. You will also sign up for a presentation slot during the week of March 2, in which your group will present the proposal to a pair of course assistants and answer questions about it.

Rubric: The proposal will be graded according to the following rubric:

Requirements checklist (all must be satisfied, otherwise the team cannot proceed with the proposed project):

> Team defines a hypothetical service into which the ML feature will be integrated
> The service will be realized using an existing open source project (at least 2.5k stars on GitHub)
> The proposed ML feature(s) will be a complementary feature
> The service will be fully hosted on Chameleon
> The proposed design involves at least one model that is trained/retrained
> Training will involve at least one high-quality non-synthetic external dataset with known lineage
> When deployed in “production”, the system will get new data and feedback from “users”, and can use this for retraining

Joint responsiblities *(3/5 points, all team members will have the same score for this part)*:

> *(0.5 points)* Describe the public-facing service that you will realize with the selected open source project (not the ML feature - the service that the ML feature will be complementary to). Discuss the audience (including anticipated number you are designing for), what their context is, etc.
> *(2 points)* Describe the design of the complementary ML feature, following the process from 1.5.5 Specifying the design, and answer questions posed by the course assistants. Make sure to discuss feedback, and how it will be used for re-training, since this is a strict requirement.
> *(0.5 points)* Describe external dataset(s) you will use, including a discussion of alignment with the proposed public-facing service. Show a few examples of real

data points, and explain the lineage of the data (who collected it, how, why). (Refer to 4.3 Acquiring training data.)

Training team member *(2/5 points)*:

(1 points) Specify the type of model(s) that will be used to realize the ML feature(s), and how they will be trained/re-trained.
(1 points) Specify input features and output.

Serving team member *(2/5 points)*:

(1 points) Estimate operational requirements for serving your ML feature, with suggested numbers (requests/second, latency/request, etc.) and justification.
(1 points) Describe how the model output(s) will translate to an *outcome* in the real system.

Data team member *(2/5 points)*:

(1 points) Describe the data flow - what data arrives at the system, how it is processed in real time for inference, how it is processed for training. (You will not specify frameworks or tools at this stage - describe what will happen to data, not how you will implement it.)
(1 points) Discuss training data more specifically, including plans for candidate selection (4.7.2 Candidate selection) and avoiding data leakage (4.7.5 Splitting and leakage)

DevOps/Platform team member (4-person teams only) *(2/5 points)*:

(1 points) Describe freshness requirements for models (how frequently, and under what circumstances, should they be retrained?) with justification, and how this will fit into your proposed automation lifecycle.
(1 points) Describe scaling requirements for the deployment (e.g. what is peak usage, what is typical usage, how will you “right size”).

Initial implementation (Due Apr 6)

Focus: Each team member delivers a runnable, role-owned subsystem on Chameleon, built to a shared interface (example payloads). Components do not need to be integrated end-to-end yet, nor integrated with the open source system.

Note that deliverables are designed so that, except for the shared items (worth 1/10 points), all team members can work independently *without* having to wait for an artifact from another team member.

Resource usage: Use Chameleon to develop your project, subject to the following:

> when you create resources (leases, server instances, volumes, object storage buckets, new security groups), you *must* include your project ID (e.g. `proj99`) as a suffix in its name. Otherwise, it will be deleted by course staff.
> follow best practices for keeping infrastructure costs low: keep compute instances alive only when you are actively working on them (*not* just to persist their data or to save on setup time), keep large data sets and model checkpoints in object storage, persist small application state data to a block storage volume, use the smallest instance type possible for the task, assign a floating IP to only one compute instance per site and use it as a "jump" host to reach others.

Some notes on advance planning:

> if you require general-purpose VMs, these generally don't require advance reservation (although even that may change if all of you try to launch many instances just before the deadline…)
> For GPU instances, the easiest types to get are `gpu_rtx_6000` at CHI@UC (NVIDIA RTX6000 x1 bare metal instance), `gpu_mi100` at CHI@TACC (AMD Instinct MI100 x2 bare metal instance), `gpu_p100` at CHI@TACC (NVIDIA P100 x2 bare metal instance) and `g1.h100.pci.1` at KVM@TACC (NVIDIA H100 x1 VM instance). You should plan access to these and make your reservation about a week in advance.
> if you require a bare metal A100 (80GB or 40GB) or A30 instance type (available on some `compute_gigaio` and `compute_liqid` nodes), these might require a little more advance planning - make reservations two weeks in advance.
> if you require a 4-GPU instance: `gpu_a100_pcie` at CHI@UC (NVIDIA A100 4x bare metal instance), `gpu_v100` at CHI@UC (NVIDIA V100 4x bare metal instance) or `g1.h100.pci.4` at KVM@TACC (NVIDIA H100 4x VM instance), you should likely plan several weeks in advance.

Format: Each team member submits different items/subject to different requirements. In the rubric,

> 📝 indicates that this item is submitted as a written document.

> 🎥 indicates you should have a short demo video showing this item live on Chameleon, from beginning to end (in faster-than-real-time or much-faster-than-real-time depending on duration).
> 📄 indicates that this item should live in your team's source code repository.
> 💻 indicates that this item should be live on Chameleon, for course staff to interact with.

Rubric: The initial implementation will be graded according to the following rubric:

Joint responsibilities *(1/10 points, all team members will have the same score for this part)*:

> 📄 A pair of JSON files representing one input sample (with real representative values) and one model output (again, with real representative values). (Training, serving, and data team members must agree on this item together; training and inference team members ingest an input like the sample and produce an output like the sample, data team member produces something like the input sample in online and offline workflows.) If your project involves more than one model, you will have one JSON pair per model.
> 📝 *(4-person team only)* A table enumerating all the containers involved in each role, with links to their Dockerfiles/Docker Compose files, and a link to the equivalent K8S manifest for each one. The object is to show that each of the individual role-owned systems will be supported by the DevOps/Platform role, although at this stage team members are working independently.

Training team member *(9/10 points)*:

Delivering a trained model file and some training code is not enough: this course is about *operationalizing* ML processes, including training. That means another engineer (or the TA) can reproduce a training run, inspect tracking information for a run, and compare candidates (including simpler baselines).

Deliverables (what you submit):

> 📝 Training runs table: a table of training runs with good candidates clearly marked; each row links to an MLflow run. An example table is below. You will highlight the rows that you consider most promising, and in the notes, explain why (e.g. one model has the best accuracy, a different model has accuracy almost as good but is much faster to train, etc.).

📄 Repository artifacts: Dockerfile(s) for the training container (and optionally, another Dockerfile for interactive development), training code as a Python script (not an interactive notebook), and sample training config file(s) if config lives in a separate file.
🎥 Sped-up demo video: one complete training run in a Docker container on Chameleon. (If training takes hours, you can record in snippets including beginning, middle, and end.)
💻 Live MLflow service running on Chameleon, browsable by course staff, with all your training runs.

Example table:

Candidate	MLflow run link	Code version	Key hyperparams	Key model metrics	Key training cost metrics	Notes
baseline	http://...	`git sha`	`lr=..., batch=..., epochs=...`	`metric1=..., metric2=...`	`wall=..., gpu_hrs=..., peak_vram=...`	establishes baseline
v1	http://...	`git sha`	`...`	`...`	`...`	tradeoff: better X, worse Y
v2	http://...	`git sha`	`...`	`...`	`...`	why it is promising + next experiment

Requirements to get credit for those deliverables:

All training runs should be executed on Chameleon from inside a container on a compute instance, and should be tracked in MLFlow. (For purposes of this project, “local” work doesn’t count.)
No one-off training scripts for different configurations: structure your code so that candidates and hyperparameters are selected via configuration (a single

configuration dictionary that you edit directly in your code, a standalone config file in JSON or YAML format that is read in by your training code, or configuration specified by command line arguments). You should have one training script (or if your model options include totally separate frameworks, like one scikit-learn and one Pytorch model, one training script per framework). (If your feature involves multiple models with different prediction tasks, then you'll have one training script per framework pre prediction task.)
For each run, you must log configuration parameters, model quality metrics that are appropriate for the prediction task, model training cost metrics (e.g. time per epoch, total training time), and information about the training environment (e.g. GPU information).
You should give your "manager" (me! I'm your manager!) choices for managing the tradeoff between training/serving speed, cost, and model quality. The table should include at least 1 simple baseline model, as well as other candidates you want to consider.
Use reasonable and well-justified strategies for hyperparameter tuning. (Note: even if you are not using Ray, there are other good libraries for hyperparameter tuning beyond grid search.)

Bonus items (to the extent that they make sense for your particular project):

> 🎥📄📝 Use Ray Train's integration with your training framework to real effect (I don't mean calling `ray submit` on an unmodified training script) in a way that goes beyond what we had done in the lab. To get bonus credit, you must show through a concrete example how your integration makes training more robus.

Additional materials for you to use:

> [Trovi artifact to bring up a "medium" VM instance with MLFlow and persistent storage](). Before you launch any training run, you should first bring up a standalone MLFlow instance with persistent storage. Then, you'll set the MLFlow training URI in your training code/environment to point to its floating IP.

Serving team member *(9/10 points)*:

The serving role must prepare a set of measured serving options (fast/good/cheap tradeoffs) so the team can choose a deployment approach during system integration.

Note that you do *not* have to wait for the training team member to deliver a trained model before you can start working! You can start developing and evaluating serving

metrics around an equivalent *un*-trained model (with base weights/random weights). Once your training teammate delivers a trained model, then you can also evaluate task quality metrics, but in the meantime you can write *code* to do so on an *un*-trained model.

Deliverables (what you submit):

> 📝 Serving options table: a table comparing multiple serving options, with the most promising options clearly marked (best options with respect to different priorities). An example table is given below.
> 📄 Repository artifacts: Dockerfile(s) for serving, serving code/serving config file(s) depending on framework, and scripts or notebooks for evaluating a serving configuration. ("Serving code" can include scripts that consume a model artifact and product an optimized model artifact, which is then served.)
> 🎥 Sped-up demo video: show your most promising serving option running on Chameleon, and responding to the agreed example request(s).

Example table:

Option	Endpoint URL	Model version	Code version	Hardware	p50/p95 latency	Throughput	Error rate	Concurrency tested	Compute instance type	Notes
baseline_http	http://...	`model id`	`git sha`	CPU	`...`	`...`	`...`	`...`	`cpu/mem`	simplest reference
onnx_or_quantized	http://...	`model id`	`git sha`	CPU	`...`	`...`	`...`	`...`	`cpu/mem`	model-level optimization
batching_or_triton	http://...	`model id`	`git sha`	GPU or CPU	`...`	`...`	`...`	`...`	`cpu/gpu/mem`	system-level

										optimization

Requirements to get credit for those deliverables:

> All experiments should run on Chameleon, from inside a container on a compute instance. (For purposes of this project, “local” work doesn’t count.)
> You must prepare and evaluate a variety of serving options, including a baseline option and some optimized options. Your optimizations should include model-level, system-level, and infrastructure-level optimizations, separately and in combination. Your evaluations should be appropriate to validate the expected benefit of each optimization, as well as its potential tradeoffs.
> Right-sizing note: for the most promising option(s), clarify CPU/memory (and GPU if any) needs using observed resource usage on Chameleon under a representative load.

Bonus items (only to the extent that they make sense for your particular project):

> 🎥📄📝 Integrate a serving framework not used in the lab (e.g. *not* FastAPI/Triton Inference Server) that improves your serving design in a meaningful way, and justify why it improves your design relative to the lab frameworks, with a concrete, realistic example. (Examples: Ray Serve, KServe, etc.)

Data team member *(9/10 points)*:

> 📝 High-level data design document. You should enumerate the data repositories (databases, lakehouses, object storage buckets, etc.) that will be used, and for each, identify:
>
> - What data is stored there, and the data schema
> - What services and processes write/update the data, and when
> - How it is versioned (data that will be used to train models must be versioned, along with enough information to track how it entered the system and how it was transformed within the system) and you should also have one or more diagrams that show the data flow.
>
> 💻 Live object storage bucket on Chameleon, browsable by course staff, with data as illustrated by your data design document.

> 📹📄 Repository artifacts + sped-up demo video: reproducible pipeline that ingests external data into Chameleon object storage, and executes whatever transformation is necessary to make it ready for training. If the data is small (less than 5GB), you should also expand the data following best practices for synthetic data generation discussed in the lecture. (Video should demonstrate everything from pipeline launch to external confirmation that it worked.)
> 📹📄 Repository artifacts + sped-up demo video: Data generator that hits the (hypothetical) service endpoints with real or synthetic data (following our best practices for synthetic data generation, as discussed in the lecture). (Video should demonstrate launch + a few minutes of runtime.)
> 📹📄 Repository artifacts + sped-up demo video: Online feature computation path for real time inference (does not have to be fully integrated with the open source service, but needs to be integrate-able…) (Video should demonstrate at least one end-to-end example.)
> 📹📄 Repository artifacts + sped-up demo video: Batch pipeline that compiles versioned training and evaluation data sets from "production" data, with well-justified candidate selection and avoiding data leakage. (Video should demonstrate everything from pipeline launch to external confirmation that it worked.)

(You are not required to use a workflow orchestrator at this stage; pipelines can be a `make` target, one-shot jobs in a Docker compose, or run with a sequence of Python scripts. But, all components must be runnable non-interactively from the artifacts saved in repository.)

Bonus items (to the extent that they make sense for your particular project):

> 📹📄📝 Integrate a data framework not used in the lab assignments, that substantially improves your data design (i.e. swapping MariaDB in place of PostgreSQL doesn't count). For example: implement a data transformation layer with `dbt`, add data quality checks with Soda, use a vector database like Qdrant, integrate a Feast feature store, implement distributed computation of features with Spark, add a DataHub data catalog. You must justify why it improves your design, using a concrete example that is realistic in the context of your proposed service.

DevOps/Platform team member (4-person teams only) *(9/10 points)*:

Deliverables (what you submit):

📝 Infrastructure requirements table: for each service running in your cluster, show the GPU, CPU, memory requests and limits you set, plus brief evidence from Chameleon showing how you arrived at appropriate values for right-sizing.
📄 Repository artifacts: IaC/CaC materials that provision Chameleon infrastructure and configure a Kubernetes cluster (cluster, networking/ingress, persistent volumes, namespaces). Also, K8S manifests and other necessary materials to deploy the open source service on which the project is based, and to deploy platform services required by other team members.
🎥 Sped-up demo video: selected open source service running inside Kubernetes on Chameleon. (Video should demonstrate everything from launching the service to confirming its health status inside K8S to validating in a browser that it is reachable and functional.)
🎥 Sped-up demo video: Shared platform services running inside Kubernetes on Chameleon, with persistent storage as appropriate. (Video should demonstrate everything from launching the service to confirming health status inside K8S to validating in a browser that it is reachable and functional.)

Requirements to get credit for those deliverables:

Kubernetes is required for 4-person teams. (For 3-person teams, Kubernetes is optional; Docker Compose is acceptable.)
Git as source of truth: IaC/CaC artifacts and Kubernetes manifests (or equivalents) are in the repo.
Durability: platform state and artifacts persist across pod restarts (MLflow artifacts and other shared artifacts use a persistent volume/object storage, not ephemeral container filesystems).
Secrets hygiene: no secrets in Git.

Bonus items (only to the extent that they make sense for your particular project):

🎥📄 Integrate a platform tool/framework not used in the lab assignments that materially improves operability, and justify why it improves your design using a realistic example. You may investigate frameworks for secrets management, TLS automation, centralized logging, image security/scanning, distributed tracing, etc. (Note: Prometheus/Grafana do not “count” because they are used in Lab 8.) You must show one concrete operational win in demo video + a short justification.

Additional materials for you to use:

> How to add a short-term GPU instance to a Kubernetes cluster, and launch GPU jobs on it.

Policy on AI use

The ML Systems Design and Operations course focuses on

> designing systems, by identifying requirements and evaluating tradeoffs and then operationalizing those designs

The cognitive work in this course is not writing code or configurations; it is making correct decisions, understanding trade-offs, defending decisions, explaining the system to stakeholders, and diagnosing failures (i.e. “making it work”). So, you are permitted to use LLMs to help write code and configs, but only as an implementation tool to help realize *your* design, not as designers.

What that means in practice for your course project is:

You own the design. You (the human) must develop the design yourself. You’ll be asked to defend your design choices, answer “what if” questions about changing requirements, and discuss tradeoffs. If you haven’t thoughtly deeply about the problem and thought through all the possibilities, you’ll struggle to do that.

LLMs may help implement your design. You can ask an LLM to help you write or modify code and configs, with the following constraints:

> *Start from the provided labs when possible*. Wherever possible, you should use the lab assignments as a starting point for code or configs (like a human would!), and build on that rather than starting from scratch. (Of course, if you are implementing something we didn’t do in the lab, you’ll do it from scratch.) This is practical (you avoid having to debug problems that I’ve already solved when developing the lab!) and it’s also realistic (in most settings, you will be modifying existing pipelines and systems, not starting a greenfield design from scratch).
> *You specify; the LLM executes*. You tell the LLM what to do, based on the design *you* developed.
> *You must understand what it produced*. You are responsible for being able to explain any code or configuration that appears in your project, including what it does and why it is needed for your design.
> *No silent design changes.* Do not allow the LLM to change configurations, parameters, or pipeline structure without your explicit decision and justification.

(This is something I have noticed they tend to do when implementing ML systems.)
Disclosure is required. Any commit that includes LLM-generated or LLM-modified code or configuration must include a lightweight disclosure (e.g. `Assisted by Codex 5.2` or equivalent).

Communication is human-only. All lab reports, project reports, project documentation, and slides must be written by you without AI assistance. This is because communicating your design is a core learning objective of this class. Only direct translation of your own writing into English (e.g., using Google Translate) is allowed.

Running systems matter, code itself doesn't. In industry, the availability of LLMs has not made ML engineering work substantially easier. Instead, it has shifted how effort is spent: less time writing artifacts (code and configurations) line-by-line from scratch, and more time specifying intent, directing tools, reviewing generated code and configurations, making corrections, and ensuring that systems are correct, robust, and operational. To the extent that this process is sometimes faster, expectations around productivity have simply increased.

In this class, similarly, expectations around outcomes must be aligned with what people can do *with* LLM assistance. In the past, producing plausible but non-operational code or configurations could serve as evidence of partial understanding of the course material. Today, that is no longer the case, because generating artifacts that look reasonable but do not run requires no expertise. Therefore, these artifacts cannot earn any credit.

What matters in this course is not the ability to produce text or code, but the ability to design, justify, and operate a real ML system. So, in this project, you are graded on:

making sound system design choices (that are aligned with business reuqirements)
justifying those choices and trade-offs using course concepts
realizing those choices in operational systems running on the course infrastructure

There is no credit for systems that are not running on Chameleon Cloud. Code or configuration that has not been executed in the target environment, or that only runs locally, does not count. Producing text is easy; making a real system run is the work.